Heat Equation with Finite Element Analysis and Real World Data Analysis

by Jack Linbeck and Joshua Dunne

May 14, 2026

Introduction

If someone were to give you along rod that is heated at one end and cooled at the other, and that person told you to provide the temperature at a particular point along the rod at a particular time, you would not be automaically able to calculate that value without some external help or algorithms. We would want to solve a partial differential equation that describes how heat diffuses through the rod, which is called the heat equation, to understand how the temperature changes over time and space.

But because this is a difficult equation to solve analytically and even more complicated to obtain a nice closed form solution, we can use an iterative method to approximate the temperature distribution along the rod at discrete points in space and time. This method allows us to solve this equation, we can use the finite element method (FEM) to discretize the spatial domain and then apply a time-stepping scheme to evolve the solution over time.

FEM is quite similar to a finite difference method, but as we deal with more complex geometries and higher dimensions, the finite element method becomes more advantageous due to its flexibility in handling irregular meshes and complex boundary conditions.

Background

The Governing Heat Equation

For a one-dimensional rod, the heat conduction is governed by:

$$\rho c_p \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2}$$

- $\frac{\partial T}{\partial t}$ = the rate of change of temperature with respect to time ($^\circ C/s$)
- $\frac{\partial^2 T}{\partial x^2}$ = sensitivity of temperature changes at a specific point from its neighbors ($^\circ C/m^2$)
- t = Time (s), • x = Position along the rod (m), • $\alpha = \frac{k}{\rho c_p}$ is the thermal diffusivity (m^2/s)
- ρ = Density (kg/m^3) • k = Thermal Conductivity ($W/m \cdot^\circ C$), • c_p = Specific Heat ($J/kg \cdot^\circ C$)

If we substitute in these units into the heat equation, we can verify that the units are consistent:

$$\left(\frac{kg}{m^3} \cdot \frac{J}{kg \cdot^\circ C} \cdot \frac{^\circ C}{s}\right) = \left(\frac{W}{m \cdot^\circ C} \cdot \frac{^\circ C}{m^2}\right)$$

$$\left(\frac{J}{m^3 \cdot^{\circ} C \cdot s}\right) = \left(\frac{J}{s \cdot m^3 \cdot^{\circ} C}\right)$$

This confirms that the units on both sides of the equation are consistent, validating the physical correctness of the heat equation in terms of dimensional analysis. This new unit is called the Volumetric Heat Capacity, which represents the amount of heat required to raise the temperature of a unit volume of the material by one degree Celsius. It is a crucial parameter in understanding how materials respond to thermal energy and is commonly used in heat transfer calculations and thermal analysis.

How do we build the heat equation from scratch?

Phase	Mathematical Logic	Summary of each Step
1. Net Energy Flow	$\frac{dE}{dt} = (E_{in} - E_{out}) - E_{loss}$	Conservation of energy: Change equals net conduction minus environmental loss.
2. Storage	$m = \rho V$ $E = mc_pT$ $\frac{dE}{dt} = \rho V c_p \frac{dT}{dt}$	Mass is density times volume. Energy storage links mass and specific heat to temperature change. Differentiating now gives us a new relationship
3. Conduction	$E_{in} - E_{out} = -V\frac{\partial}{\partial x}(E_{flux})$	Net heat movement is the spatial change of energy flux over the slice volume V.
Fourier's Law	$E_{flux} = -k\frac{\partial T}{\partial x}$	The energy flux, representing flow intensity, depends on the material's thermal conductivity k.
Net Result	$E_{in} - E_{out} = Vk\frac{\partial^2 T}{\partial x^2}$	This result gives a second derivative (curvature), which forces heat to spread away from hot peaks
4. Cooling	$E_{loss} = V\nu(T - T_\infty)$	Energy loss to air ($T_\infty = 20.8$) using the effective cooling coefficient ν.
5. Substitution	$\rho V c_p \frac{dT}{dt} = Vk\frac{\partial^2 T}{\partial x^2} - V\nu(T - T_\infty)$	Substitute the movement and loss terms back into the original balance equation.
6. Final PDE	$\frac{dT}{dt} = \alpha\frac{\partial^2 T}{\partial x^2} - \nu(T - T_\infty)$	Divide by $\rho V c_p$ to solve for $\frac{dT}{dt}$ using diffusivity $\alpha = \frac{k}{\rho c_p}$, which produces our Heat Equation!!

Model Development

Boundary and Initial Conditions

The model uses the Dirichlet boundary conditions. Meaning the temperature never changes at the boundaries yet allows every other value to vary. The specific conditions are:

$$T(0,t) = 100^{\circ}\text{F} \quad \text{(Left Boundary)}$$
$$T(L,t) = 20^{\circ}\text{F} \quad \text{(Right Boundary)}$$
$$T(x,0) = T(x) \quad \text{(Initial Temperature Gradient)}$$

Finite Element Analysis

The rod is divided into n equally spaced elements of length $h = L/n$. This discretization allows us to approximate the continuous temperature distribution along the rod using a finite number of points. But this results in a system of equations that can be solved using numerical methods.

1: Spatial Discretization (FEM)

For each element along the rod, the local stiffness or local connectivity matrix, $\mathbf{K}^{(e)}$, how each element connects to its neighbors and transfers heat, is defined as:

$$\mathbf{K}^{(e)} = \frac{k}{h}\begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}$$

2. Transient System (Backward Euler):

To ensure unconditional stability, an implicit scheme is used:

$$(\mathbf{I} - \alpha\Delta t\mathbf{K}')\mathbf{T}^{n+1} = \mathbf{T}^n + \mathbf{b}_{bc}$$

Let's just quickly derive this Backward Euler scheme for our transient heat equation. Starting with the heat equation:

$$\frac{\partial T}{\partial t} = \alpha\frac{\partial^2 T}{\partial x^2}$$

We can discretize the time derivative using a backward difference approximation and our spatial second derivative using the finite element method from above rather than a finite difference method. The backward difference approximation for the time derivative is:

$$\frac{T^{n+1} - T^n}{\Delta t}\mathbf{I} = \alpha\mathbf{K}^{(e)}$$

Rearranging this gives us to solve for the temperature at the next time step given the current temperature distribution:

$$T^{n+1}\mathbf{I} = T^n\mathbf{I} + \alpha\Delta t\mathbf{K}^{(e)}$$

$$(\mathbf{I} - \alpha\Delta t\mathbf{K}')\mathbf{T}^{n+1} = \mathbf{T}^n$$

Now, we can express the spatial second derivative using the finite element method, which leads to a system of equations that can be represented in matrix form as:

$$\frac{dT}{dt} = \alpha\frac{\partial^2 T}{\partial x^2} - \nu(T - T_\infty)$$

$$\frac{T_i^{n+1} - T_i^n}{\Delta t} = \alpha\left(\frac{T_{i+1}^{n+1} - 2T_i^{n+1} + T_{i-1}^{n+1}}{(\Delta x)^2}\right) - \nu(T_i^{n+1} - T_\infty)$$

$$\text{Let F} = \alpha\frac{\Delta t}{(\Delta x)^2}$$

$$-FT_{i-1}^{n+1} + (1 + 2F + \nu\Delta t)T_i^{n+1} - FT_{i+1}^{n+1} = T_i^n + \nu\Delta tT_\infty$$

$$\begin{bmatrix} (1+2F+\nu\Delta t) & -F & 0 & \dots \\ -F & (1+2F+\nu\Delta t) & -F & \dots \\ 0 & -F & (1+2F+\nu\Delta t) & -F \\ \vdots & \vdots & -F & (1+2F+\nu\Delta t) \end{bmatrix} \begin{bmatrix} T_1^{n+1} \\ T_2^{n+1} \\ T_3^{n+1} \\ \vdots \\ T_N^{n+1} \end{bmatrix} = \begin{bmatrix} T_1^n + \nu\Delta t T_\infty + FT_L \\ T_2^n + \nu\Delta t T_\infty \\ T_3^n + \nu\Delta t T_\infty \\ \vdots \\ T_N^n + \nu\Delta t T_\infty + FT_R \end{bmatrix}$$

3. Boundary Treatment (Dirichlet):

Now, accounting for the initial Dirichlet boundary conditions, we can incorporate the boundary conditions into the system of equations by adding a vector $\mathbf{b}_{bc}$, which enforces the fixed temperatures at the boundaries ($100°\text{F}, 20°\text{F}$), effectively driving the heat flow toward a linear steady-state where $\mathbf{KT} = \mathbf{0}$. meaning there is no net heat flow and the temperature distribution is stable.

$$(\mathbf{I} - \alpha\Delta t\mathbf{K}')\mathbf{T}^{n+1} = \mathbf{T}^n + \mathbf{b}_{bc}$$

4. Examples of Space vs Time Discretization:

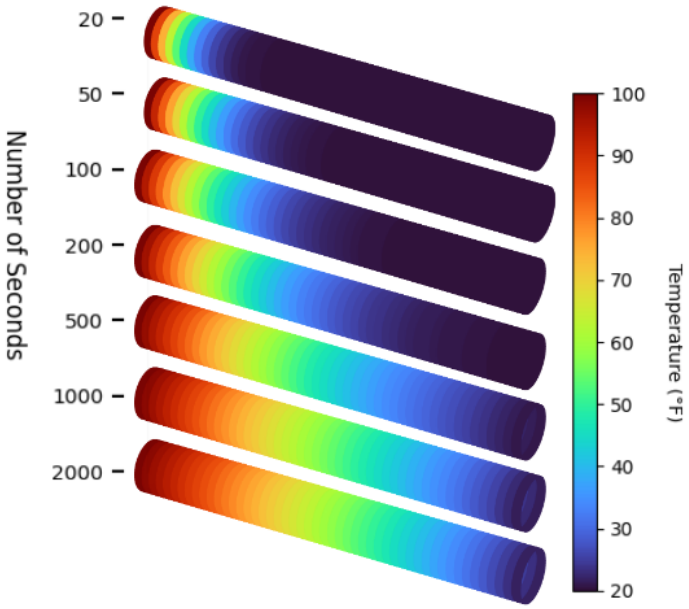


Figure 1: Time Increase as space slices are frozen

- We set up the boundary conditions: 100F on left, and 20F on right
- Define a fixed value of space slices, in this case, exactly 50 slices, to measure the temps
- Allow time to increase from 20 to 2000 seconds
- We can figuritively hit play on the simulation and see how the heat propigates along the tube
- Notice we do eventually reach an approximate steady state solution
- The resolution of the data is constant but this helps us "see" how the heat flows along the tube until it reaches the cold end

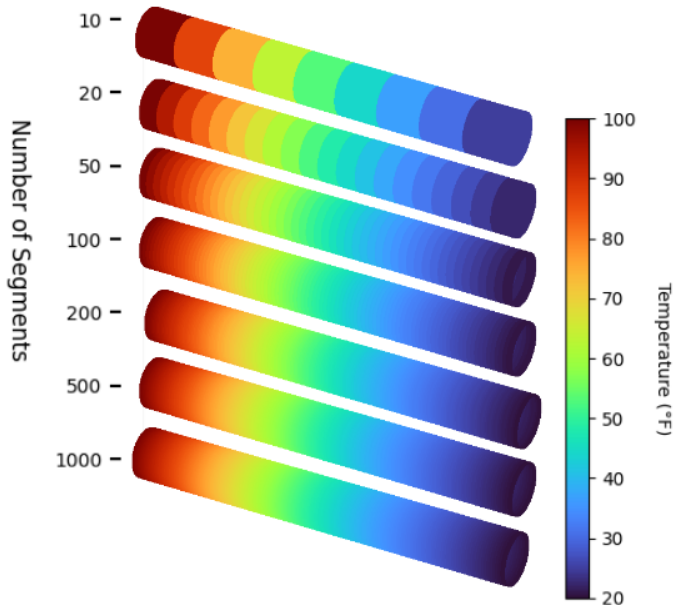


Figure 2: Space slices increase as time is frozen

- We set up the boundary conditions: 100F on left, and 20F on right
- Define a fixed distribution of temperature, in this case, frozen at exactly 500 seconds, to measure the temps
- Allow the space slices to increase from 10 to 1000 seconds
- We can figuritively hit play on the simulation and see the quality and accuracy of the model increase
- Notice we do eventually reach the same approximate steady state solution
- The temperature of the data is constant but this helps us "see" how taking more slices along the tube can give us better accurate readings

Analysis and Results of General Model

One key takeaway is that having many many time points to measure and many many space points to measure can greatly help get the best data from the heated tube along with seeing the flow progression better. Computationally, its not ideal to have lets say making 1000 time steps along with 1000 space steps, since, roughly, thats a million data values in the matrix system. Not super sure on that but you get the idea. Unless we plot a 3D graph of time vs space vs temperature, its not easy to get all of the information all at once, so its best to measure one "dimension" at once.

Because this is the general model, we can substitute any values to any of the constants we could theoretially calculate and plot something very unrealistic, but we have full control of everything. This can be bad because unless we knew a good range for every constant, just guessing and hoping for something realistic is quite difficult. When I was messing around with α, I made it negitive, positive, 1, 10,1000, 0.0005, etc but as we will see later, a good approximate value is about 0.022, which we later gather from actual real world data. So lets grab some and see what we can do with it

Real World Data Comparison

Air Temperature Affect

To make this equation more realistic by incorporating ambient air influence, the 1D heat conduction is now governed by:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} - \nu(T - T_\infty)$$

- ν: Convection Coefficient (s^{-1})
- T = Temperature (°C),
- T_∞ = Ambient Temperature (°C)

If we convert this to a similar backward Euler scheme, we can derive the following scheme:

$$(\mathbf{I} - \alpha\Delta t\mathbf{K}' + \nu\Delta t\mathbf{I})\mathbf{T}^{n+1} = \mathbf{T}^n + \nu\Delta t T_\infty \mathbf{I} + \mathbf{b}_{bc}$$

This now makes the iteration scheme a bit more complicated than before, yet all we've actually done is add more constants to both sides of the equation to involve the ambient temperature.

Using a Real World Dataset

We are going to use a pretty large dataset of temperature measurements along a rod over time, which was collected by students at the Missouri University of Science and Technology. This dataset also provides extra parameters for different types of metals and a setting for including wind moving the air outside of the rod.

Important mention: In the main corner models, we had a similar number of space and time points, but this model has way more time points to compensate for barely any space points

From what we mentioned before, this dataset is unique because the experiement only had 6 points along the rod that measureed the temperateure, but they had 4000 time steps, which compensates for the few spatial points and allows us to have a more accurate model with such a high degree of precision.

What is this provided code doing?

First off, Magic. Secondly, we were given a list of material properties and respective settings

- Aluminum (Al):
  Density = 2700 kg/m^3, Thermal Conductivity = 205 $W/m \cdot^\circ C$, Specific Heat = 900 $J/kg \cdot^\circ C$
- Stainless Steel (SS):
  Density = 8000 kg/m^3, Thermal Conductivity = 16 $W/m \cdot^\circ C$, Specific Heat = 500 $J/kg \cdot^\circ C$
- Copper (Cu):
  Density = 8900 kg/m^3, Thermal Conductivity = 385 $W/m \cdot^\circ C$, Specific Heat = 385 $J/kg \cdot^\circ C$
- 3D Printed Stainless Steel (ADD):
  Density = 8000 kg/m^3, Thermal Conductivity = 16 $W/m \cdot^\circ C$, Specific Heat = 500 $J/kg \cdot^\circ C$
  (but with different thermal properties based on printing direction and cooling fan air flow)

Material and environmental settings for the dataset:

- Temp = Bool: Allows for a custom initial temperature instead of starting at 0F
- Wind = Bool: Allows for wind to push and affect the temperature outside of the rod,
- Al = Bool: Indicates the use of aluminum,
- SS = Bool: Indicates the use of stainless steel,
- Cu = Bool: Indicates the use of copper,

3D printed Stainless Steel settings for more nuanced thermal properties:

- ADD = Bool: Indicates the use of 3D printed Stainless Steel
- XN = Bool: Printing the rod along the X-negitive direction
- XP = Bool: Printing the rod along the X-positive direction
- YN = Bool: Printing the rod along the Y-negitive direction
- YP = Bool: Printing the rod along the Y-positive direction

These setings are a bit confusing at first, but due to the nature of 3D printing, depending on the leading direction of printing, the cooling fan, can affect extrution and the drying progress so the densitiy and thermal properties of the rod are slightly different.

Depending on the options we choose, the code will pull an excel file provided by those students that are about 4000 rows by 8 columns of data. Then the code compiles it together and from the best of my understanding, does the following:

1. **Aquiring Data**: Pulls all the temperature data at the 6 points along the rod over time given our chosen settings
2. **Building the Matrix Model**:Fits all the needed data into a PDE approximating matrix along with the boundary conditions and the initial temperature gradient
3. **Boundary Gradient Approximation**:Iteratively applies a second order backward finite difference scheme best approximate points at and near the boundaries. This solves for $\frac{\partial T}{\partial x}$ at the boundaries to know how much heat enters the rod.
4. **Steady State Solution Approximation**:Then approximates the points near the end of the time range for the steady state solution. This solves for ν (convection coefficient) and an initial guess for k (thermal conductivity) for the material and settings we chose
5. **Transient Modeling**:Inputs the predicted temperatures into a 100 term fourier series to build a nice continuous function from distrete points. This solves for a very good approx for α (speed of the heat) and k (conductivity).
6. **Optimization**:Compares the predicted temperatures with the actual temperatures using a least squares error metric to see how well the model is doing. This solves for the error of the model and how well it fits the real world data.
7. **Gathering Parameters**: This Magic Code found a great approximation for the following values: ν, k, and α and $\frac{\partial T}{\partial y}$ at the boundaries, now we can substitude these values back into the PDE
8. **Visual Comparison**:Then plots the predicted and actual temperature values over time to visually compare the model's performance against the real-world data.

Honestly, it's a bit unfortunate I cant easily show all 500 lines of code, and i havent quite understood entirely what its exactly doing, but i at least figured out the general steps. I hope that after this semester ends I can look

back and dive deeper and really pick it apart because its fasinating.

Summary of the Main important steps

1. Backwards Euler to find all the datapoints:

$$(\mathbf{I} - \alpha \Delta t \mathbf{K}')\mathbf{T}^{n+1} = \mathbf{T}^n + \mathbf{b}_{bc}$$

2. Fourier Series to build Continuous function:

$$f(x) = \frac{a_0}{2} + \sum_{n=1}^{100} a_n \cos(nx) + b_n \sin(nx)$$

3. Least Squares to compare model vs data:

$$S = \sum_{i=1}^{n} (y_i - f(x_i))^2$$

What does the code tell us?

Symbol	Parameter	Value	Units
ρ	Density	7828.79	kg/m^3
c_p	Specific Heat Capacity	502	J/(kg·C)
k	Thermal Conductivity	11.313	W/(m·C)
ν	Kinematic Viscosity	0.096	m^2/s
α	Thermal Diffusivity	0.224	m^2/s
T_∞	Ambient Temperature	20.8	°C

Table 2: Summary of Material Constants and Environmental Parameters

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} - \nu(T - T_\infty) \Longrightarrow \rho c_p \frac{\partial T}{\partial t} = k \frac{\partial^2 T}{\partial x^2} - \nu(T - T_\infty)$$

$$\frac{\partial T}{\partial t} = 0.022387 \frac{\partial^2 T}{\partial x^2} - 0.096192(T - 20.8)$$

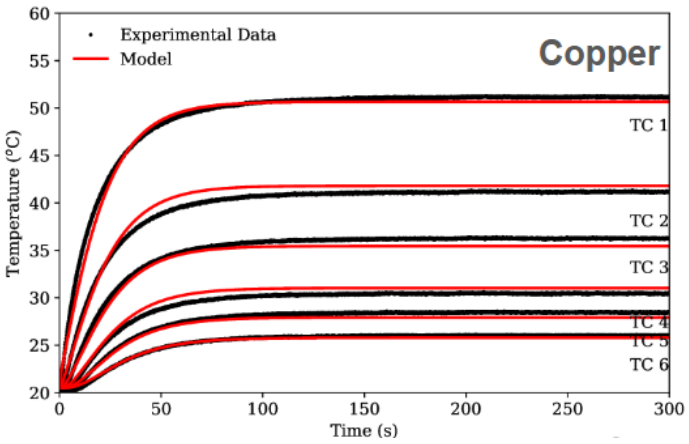
Thus, this is our final Heat Equation with everything plugged in, and now we can use this to find temperature values between the data points and take derivates and actual use math to explore other details and info rather

How accurate is our model?

Quite accurate! The images below show that both plots look nearly identical, which I will explain more in a bit. Honestly surprised we could take 4000 points per thermocouple and get such a good fit with only 6 spatial

points considering that we found out more time and more time points are always good. But the 4000 time points defintally compensate for the lack of spacial thermocouple points. This is important to note because realistically, measuring temperature is simple but strapping 10, 20, 50 thermometers on a 15 cm metal rod seems very difficult, yet we can tell a computer to get a reading every 100th of a second much easier, so that was probably the best way to go about this. The code also outputs a highly detailed chart of all the obtained values, with standard error, the unknown parameter estimation errors, and more.

Analysis and Results of Real World Model



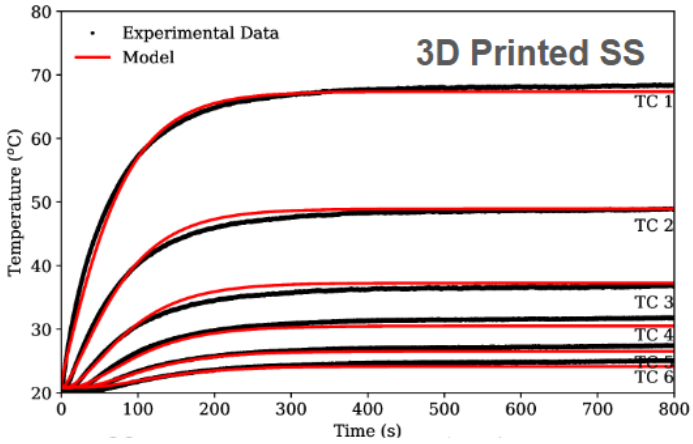


Table 3: Thermal Parameter Comparison: Bulk Copper vs. 3D-Printed Steel (XN)

Property	**Copper**	**3D Printed**	**Physical Interpretation**
Conductivity (k)	384.72 W/m·°C	14.40 W/m·°C	3DP is ≈ 26x more resistive to heat flow than Cu.
Diffusivity (α)	0.0738 m^2/s	0.0213 m^2/s	Cu reaches steady state ≈ 3.5x faster than 3DP.
Convection (h)	165.41 $W/m^2 \cdot^\circ C$	128.70 $W/m^2 \cdot^\circ C$	Variations due to ambient lab/wind conditions.
Avg. Std. Error	0.95	0.73	Low error validates 1D FEA model robustness.

Another way to confirm this code and model is correct is that the conductivity of copper *should* in fact be more than 3D printed Stainless Steel, same goes for Diffusitivity and Convenction from what we know of metal properties so that checks out. Also, from the images above, all 1500 data points per thermocouple are barely visible, but again, remember they are just discrete points. With this algorithm, we build the red model graphs, which look super accurate for both, which the bonus of being continuous everywhere. We can note that a 3D printed object *should* also be more uniform in terms of density, material composition, imperfections, and heat transfer thus we should expect the average standard error to be smaller. Metal poured into a cast to be shaped would probably be cooled ununiformly and could leave imperfections as it solidifies based on many factors too, just imagine the differnece between the internal structual composition of a tower made of wood vs plastic. Even if the analogy doesnt hit as well as i want it to, we can still imagine and feel how different they are. As we said before, we had settings for regular stainless steel and aluminum and we produce a similar very accurate graph.

Discussion

Conclusions

Post Presenting Notes

Someone brought up a closed solution form for the solved or general heat equation. I completely forgot to even consider that, since obviously we can't just have a complete PDE and immediately plot it. I also want to dive deeper in the code since I was asked how exactly the results of backwards euler gets subbed into the fourier series, since we need a function of some kind to build the coefficients of the series. It would also be nice to check on global and local stability or other applications from class. Also, having a farther apart central/backwards finite difference could lead to more accurate data too, like for some reason what if we have a gap in data or we want less time steps, or various other reasons. Later on, I also want to write down and compare computation time for every code example and future ones. Even though my computer, can, I think, spit out all of these plots pretty quick, I would love to record how the calculation time changes as we increase time steps and space steps in orders of magnitudes, and the finite difference approximation methods, etc. Overal I'm super glad I went down the route of using real world data since that helped strengthen our project and its something better and more interesting to latch on to and learn.

Reflection (1 Page)

- Include a short reflection addressing:
- What feedback did you receive on your poster?
- What did you improve in the presentation and paper?
- What mathematical aspects did you deepen?
- What did you learn about communicating applied math?

Audio Feedbackn from Poster

1. well executed focus and application
2. stronger applied poster in the class
3. bit of visual clutter (images too big) and more story
4. clear images but print whole poster
5. great font sizes to see from a distance
6. main thing is add more detailed images and less frozen either and hard to itentify final result since they look too similar
7. great analysis of heat equation and details behind them and how the method works

8. really like i took olivers suggestion for real world data thats a strong element of the poster and great contant i can keep leaning on
9. main take away is clear but modify the first parts of story and last part is great
10. what did we learn feedback, a box for the take away, not obvious for conclusion for reader but great understanding
11. need to show off the references more in poster but pretty good 98

References (1 Page)

General Links for Sources about FEA

- **"Why is a Triangular Element Stiffer?"** *EnterFEA.* https://enterfea.com/why-is-a-triangular-element-stiffer/
- **"The Importance of Mesh Convergence."** *NAFEMS Knowledge Base.* https://www.nafems.org/publications/knowledge-base/the-importance-of-mesh-convergence-part-1/

- **"What is Finite Element Analysis and How Does It Really Work?"** *Fidelis FEA*. https://www.fidelisfea.com/post/what-is-finite-element-analysis-and-how-does-it-really-work
- **"What Is FEA — Finite Element Analysis?"** *SimScale Documentation*. https://www.simscale.com/docs/simwiki/fea-finite-element-analysis/what-is-fea-finite-element-analysis/
- **"What Is Finite Element Analysis?"** *MathWorks Discovery*. https://www.mathworks.com/discovery/finite-element-analysis.html
- **"Solving Partial Differential Equations with Finite Elements."** *Wolfram Reference*. https://reference.wolfram.com/language/FEMDocumentation/tutorial/SolvingPDEwithFEM.html

Links for Thermal Property Values Estimator Infomation and Coding

- **"Material thermal properties estimation via a one-dimensional transient convection model."** *Applied Thermal Engineering*, ScienceDirect. https://doi.org/10.1016/j.applthermaleng.2020.116230
- **"Data on the Validation to Determine the Material Thermal Properties Estimation."** *Data in Brief*, ScienceDirect. https://doi.org/10.1016/j.dib.2021.107577
- **"Python code for 1D Multiple Parameter Estimator."** *Zenodo Repository*. https://zenodo.org/records/5683312
- **"oneDkhEstimator Required Dataset."** *Mendeley Data*. https://data.mendeley.com/datasets/sbcf36976g/1
- **"Link to the Magic code directly from my Github Repository"** *Jack's Github Files* https://github.com/JackL1999/Math-675-Grad-DIff-EQ-Project/blob/main/Project%20Online%20Data/oneDkhEstima py
- **"Gemini 3 Flash."** *Google AI*. https://gemini.google.com. Used for generating 3D tube diagram code, used for understanding the provided magic code.

Links to Presentation and Poster

- **Presentation Slides.** *Google Slides*. https://docs.google.com/presentation/d/1VPHus1hrQ4LEmZuCRAmAq4DViZxpAG3NeTNOqDCx5iA/edit?usp=sharing
- **Project Poster.** *Google Slides*. https://docs.google.com/presentation/d/12PqFTOBQ9jGK5aHqCC0VSaQxrgSarbEYJ4f69zTSPOY/edit?slide=id.p#slide=id.p